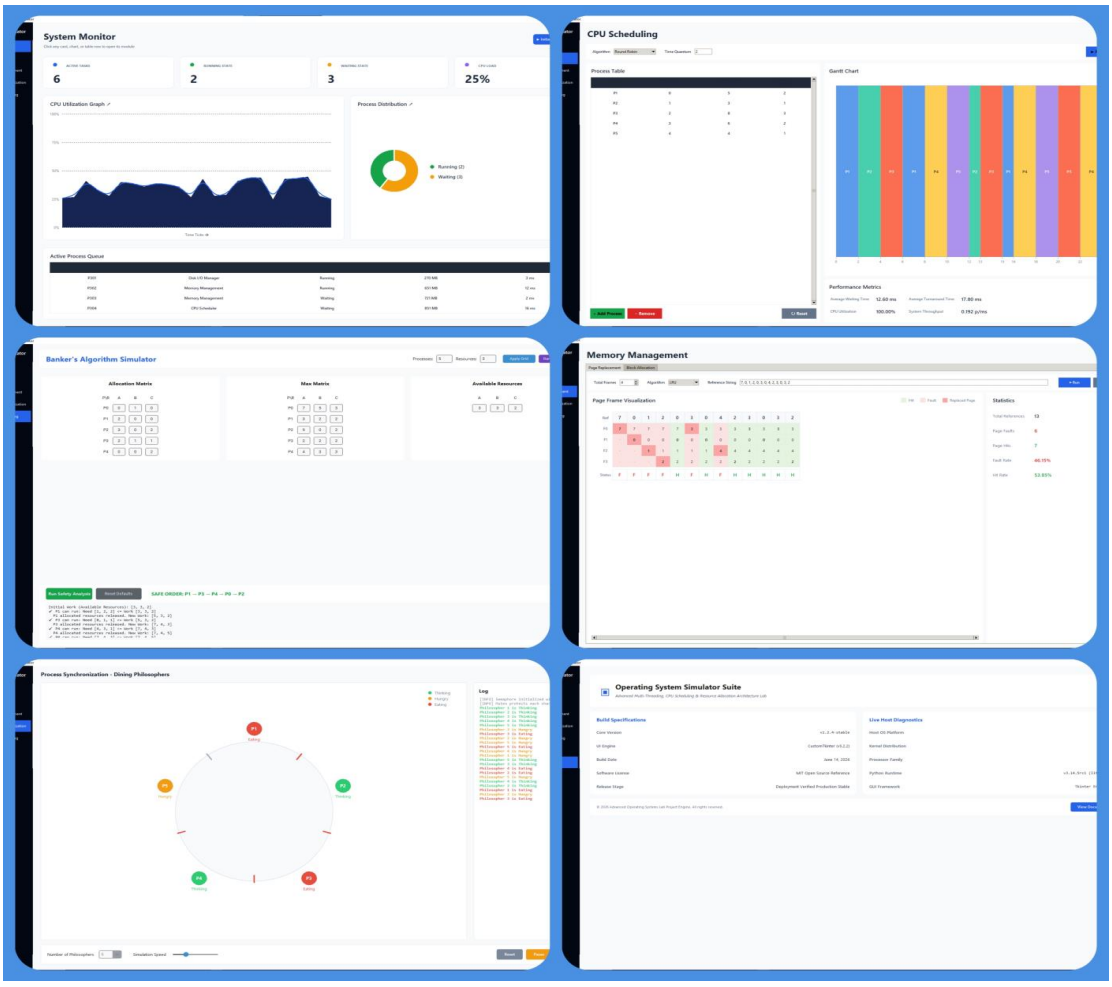
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Interactive Operating System Simulator

Assignment Project Report

Course Title	Operating System and System Programming
Course Code	CSE 3203
Assignment Title	Interactive Operating System Simulator



ID	Name
232311131	*OLOC Chandro (Partha)*
232311136	Partho Kmar Ray
232311139	*Md.AKIB Jabed*

Submitted To,

Md.Fyzul Islam (lecturer Dept. Of CSE VU) Submission Date : 15 –05 -2026

Executive Summary

Mini Operating System Simulator is an interactive educational application built on a desktop basis, developed by means of the Python programming language with the use of the Tkinter graphical user interface package. The application was intended to connect theoretical knowledge related to computer science to the real operations within the system, which makes it possible to visualize different behaviors of the system. Instead of learning the behavior passively, one can observe it actively by performing certain actions with the program itself.

Project Outcome

A functional, modular, and visually coherent simulator that can be demonstrated during a viva, lab presentation, or assignment defense. Every screenshot in this report was captured from the actual running program.

Table of Contents

1.Requirements and Compliance

The implementation was planned directly from the provided specification. Although the requirement states that at least three modules must be implemented, this project implements every listed module to provide a complete demonstration platform.

Module	Required Concept	Implementation	Status
CPU Scheduling	At least two algorithms, Gantt chart, waiting and turnaround time	FCFS, SJF, Round Robin, Priority; Gantt chart and metrics	Completed
Memory Management	Any two techniques, visualization and fragmentation	First Fit, Best Fit, Worst Fit; allocation and fragmentation view	Completed
Page Replacement	FIFO, LRU, Optimal; faults, hits and frame table	All three algorithms with colored frame-table visualization	Completed
Synchronization	One classic problem using semaphore/mutex/monitor	Dining Philosophers with mutex/semaphore concepts and animated states	Completed
Deadlock Handling	Banker's Algorithm, safe sequence and allocation table	Need matrix, safe-state test, safe sequence and deadlock result	Completed

2. Project Objectives:

The main aim of this research project is to create an effective learning aid that would link theory of computer science and engineering systems. The engineering goals for the design of this project include:

- **Algorithmic Execution & Simulation:** Translate abstract, low-level operating system paradigms and algorithms into fully operational, executable visual simulations.
- **Interactive Parameter Modeling:** Empower users with dynamic input controls to manipulate algorithmic variables on the fly, providing immediate, deterministic visual feedback.
- **High-Fidelity Visualization:** Demystify complex system behaviors—including CPU scheduling, memory management, process synchronization, deadlock states, and storage allocation—through clean, real-time graphical interfaces.
- **Modular & Scalable Architecture:** Engineering the codebase using strict modular design principles, ensuring that individual algorithms can be independently isolated, rigorously tested, and scaled without architectural regressions.
- **Analytical Evaluation & Performance Metrics:** Compute and display comprehensive, real-time performance metrics to facilitate deep technical analysis, cross-algorithm comparison, and robust support for academic evaluations and *viva voce* defense presentations.

3. Technology and System Architecture

Component	Technology / Purpose
Programming Language	Python 3
GUI Framework	Tkinter and ttk widgets
Image Support	Pillow for screenshot export
Algorithm Layer	Pure Python functions for scheduling, page replacement, Banker's Algorithm, and allocation
Visualization Layer	Tkinter Canvas, Treeview tables, labels, cards, and status logs
Application Structure	Single desktop application with independent navigation modules

3.1 Layered Design

USER INTERFACE LAYER

Sidebar navigation, forms, buttons, tables and visual controls

↓

SIMULATION / ALGORITHM LAYER

Scheduling, page replacement, allocation, synchronization and deadlock logic

↓

OUTPUT AND VISUALIZATION LAYER R

Gantt chart, frame table, resource table, status animation, metrics and disk blocks

4. Dashboard and User Interface

The application utilizes a modern, high-contrast design language featuring a sleek, dark sidebar navigation panel paired with a clean, highly legible workspace. This visual hierarchy isolates system controls from data visualization areas, minimizing cognitive load for the user.

Acting as the central hub of the application, the primary dashboard aggregates critical system telemetry in real time, including:
System Metrics: Live CPU utilization and total process counts.

Thread Telemetry: Real-time tracking of active (running) and blocked (waiting) process states.

Impact on Design: The design creates an immersive experience similar to that of an OS control center by integrating all different data streams into one interface. It provides the users easy access to all six simulation modules through this single window, without interrupting their tasks.

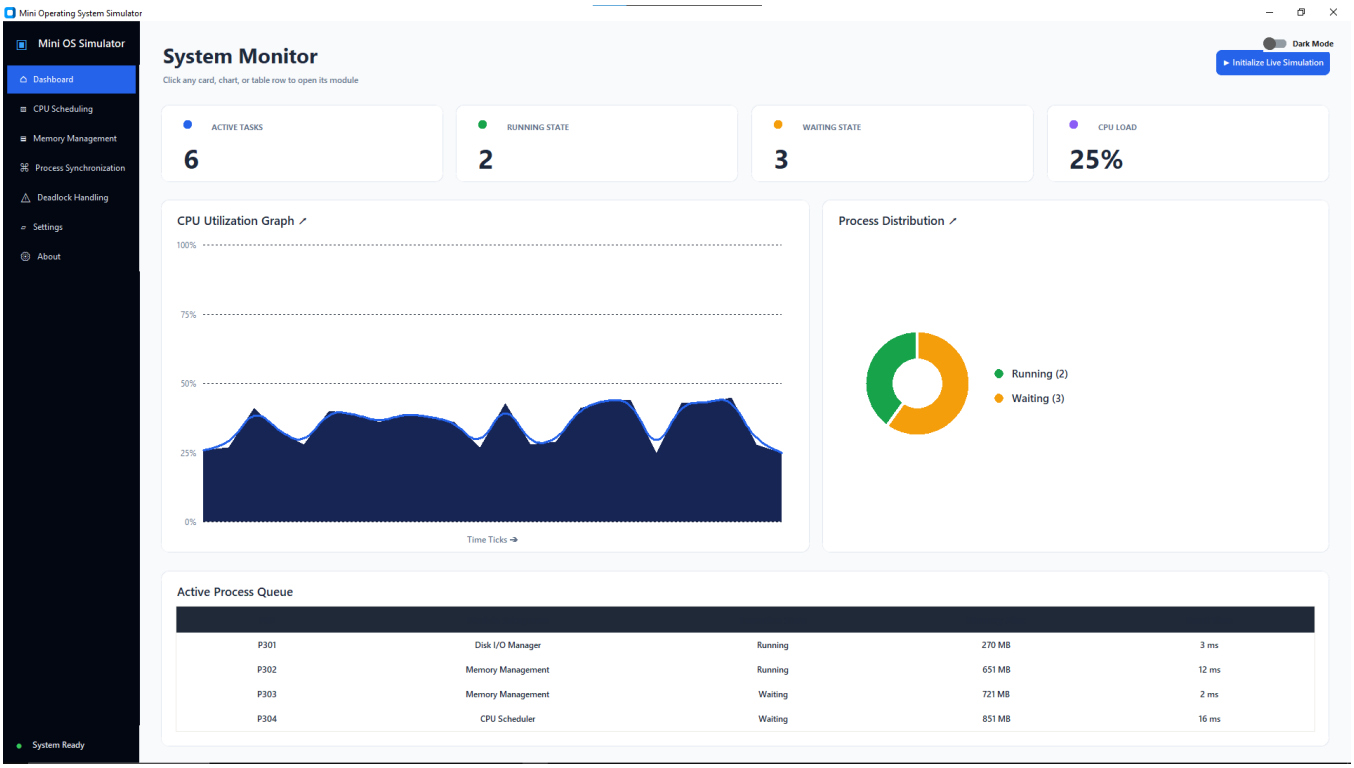


Figure 1: Main dashboard of the Mini Operating System Simulator

The dashboard is not only decorative. It demonstrates how an operating system presents aggregated monitoring information: active process count, process state, CPU load, and recent process attributes. The navigation system also helps keep the code modular because each menu item loads a dedicated simulation view.

5. CPU Scheduling Simulator

The CPU Scheduling module accepts Process ID, Arrival Time, Burst Time, Priority, and—when Round Robin is selected—Time Quantum. It then generates a Gantt chart and calculates waiting time, turnaround time, average values, CPU utilization, and throughput.

5.1 Implemented Algorithms

- First-Come, First-Served (FCFS): executes processes in arrival order.
- Shortest Job First (SJF): chooses the ready process with the smallest burst time.
- Round Robin (RR): assigns each ready process a fixed time quantum in cyclic order.

- Priority Scheduling: chooses the ready process with the highest priority; a lower numeric value represents higher priority in this implementation.

Scheduling Metrics

Turnaround Time = Completion Time − Arrival Time

Waiting Time = Turnaround Time − Burst Time

CPU Utilization = Busy CPU Time ÷ Total Elapsed Time × 100%

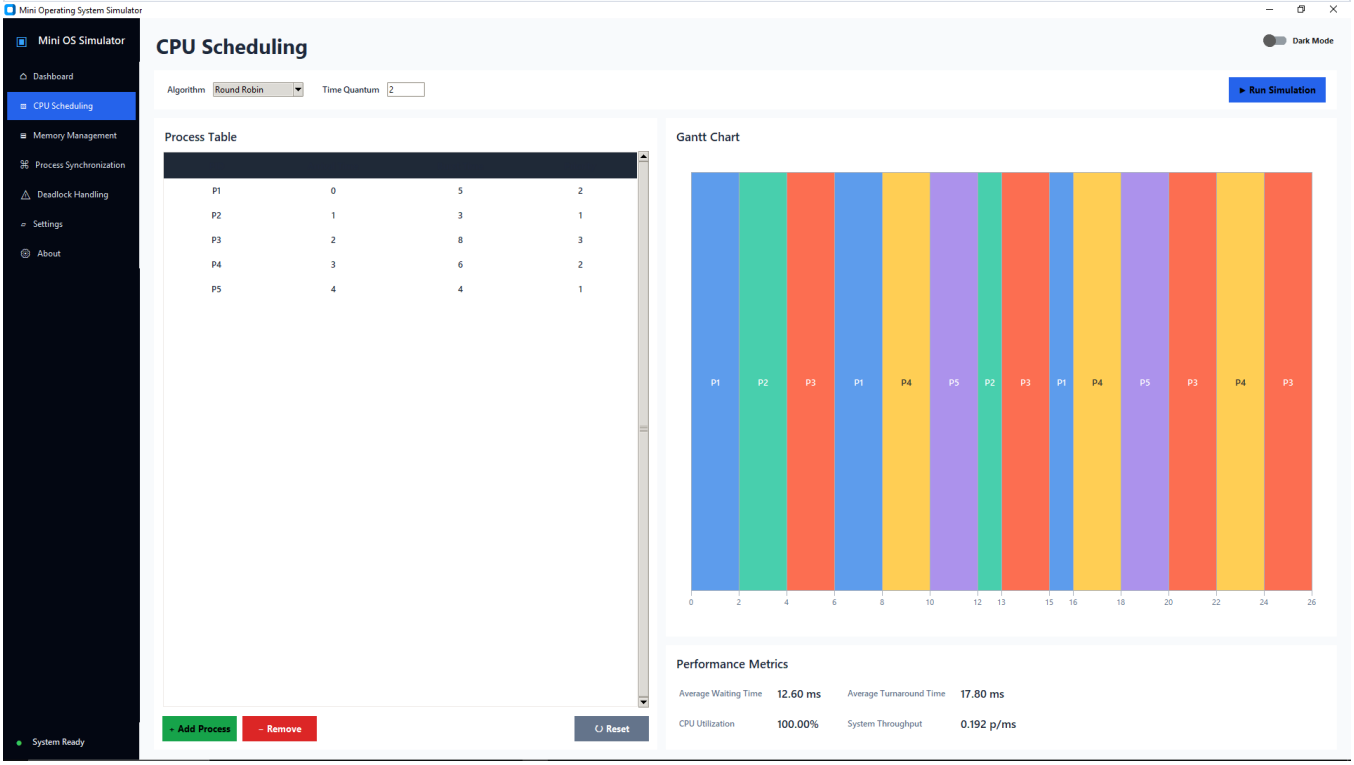


Figure 2: Round Robin scheduling with a generated Gantt chart and performance metrics

5.2 Processing Logic

1. Read all process attributes from the process table.
2. Select the scheduling function according to the chosen algorithm.
3. Generate execution segments containing Process ID, start time, and end time.
4. Calculate completion, turnaround, and waiting time for each process.
5. Render the segments proportionally on the Gantt-chart canvas and display aggregate metrics.

6. Memory Management and Page Replacement

6.1 Contiguous Memory Allocation

The Block Allocation tab implements First Fit, Best Fit, and Worst Fit. A process is assigned to a memory block only when the remaining block size is large enough. The simulator displays allocated processes, unallocated processes, remaining space, and total external fragmentation.

- First Fit selects the first block large enough for the process.
- Best Fit selects the smallest suitable block to minimize immediate unused space.
- Worst Fit selects the largest available block in an attempt to leave a sizeable remainder.

6.2 Page Replacement

The Page Replacement tab accepts the number of page frames and a reference string. FIFO, LRU, and Optimal replacement are available. Every column in the frame table represents one page reference; green columns indicate hits and red columns indicate page faults.

- FIFO removes the page that entered memory earliest.
- LRU removes the page that has not been used for the longest past interval.
- Optimal removes the page whose next use is farthest in the future; it provides a theoretical minimum fault count.

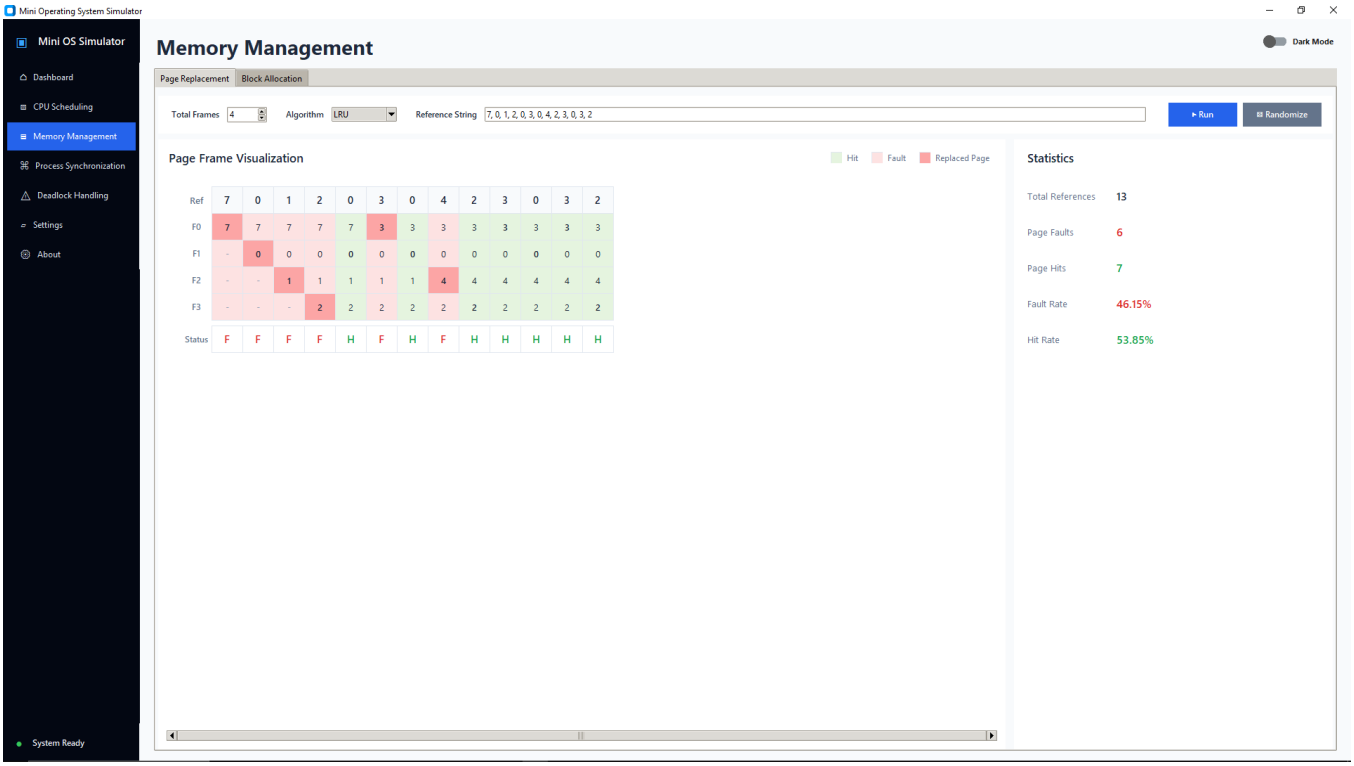


Figure 3: LRU page replacement with frame-table, fault, hit, and rate visualization

Displayed Statistics

Total references, page-fault count, page-fault rate, hit count, and hit rate are calculated after every simulation run.

7. Process Synchronization

The **Process Synchronization Module** provides a high-fidelity visual simulation of the classic Dining Philosophers problem to model concurrency challenges. The scenario maps five concurrent processes competing for a finite set of shared, overlapping resources. Each process dynamically transitions through a lifecycle of distinct operational states: *Thinking*, *Hungry*, and *Eating*. By treating resource acquisition as a mutually exclusive operation, the system practically illustrates the underlying mechanics of mutex locks and semaphores.

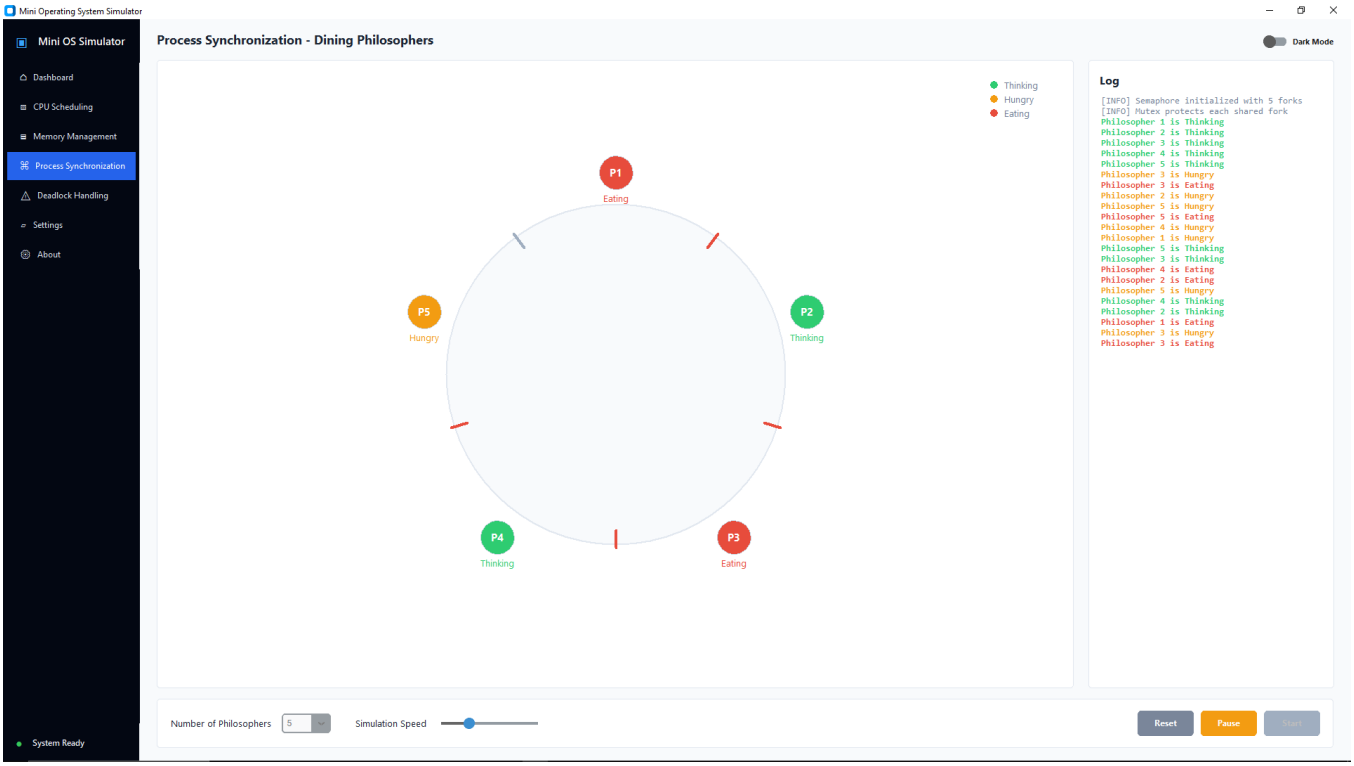


Figure 4: Dining Philosophers visualization with states, shared forks, controls, and event log

7.1 Deadlock-Free Strategy

The simulation enforces mutual exclusion in that it does not allow adjacent processes to simultaneously enter the eating state.

A philosopher can enter into the Eating state only after acquiring both localized resources.

It is demonstrated through the process of state transitions, resource hierarchy, and allocation discipline that the circular wait problem can be solved.

- Mutex concept: only one philosopher can own a specific fork at a time.
- Semaphore concept: shared-resource availability controls entry into the critical section.
- Critical section: the Eating state, because two shared forks are held simultaneously.
- Log output: records state changes and resource-acquisition events for explanation and debugging.

8. Deadlock Handling

Deadlock Avoidance module has an interactive application of the Banker's algorithm to create a safe state for the system and prevent deadlock.

The evaluation engine takes in user-defined inputs for vectors of Available resources, Matrix of Maximum demands, and the Allocation Matrix.

It computes the need matrix through a standard matrix equation Need = Maximum – Allocation.

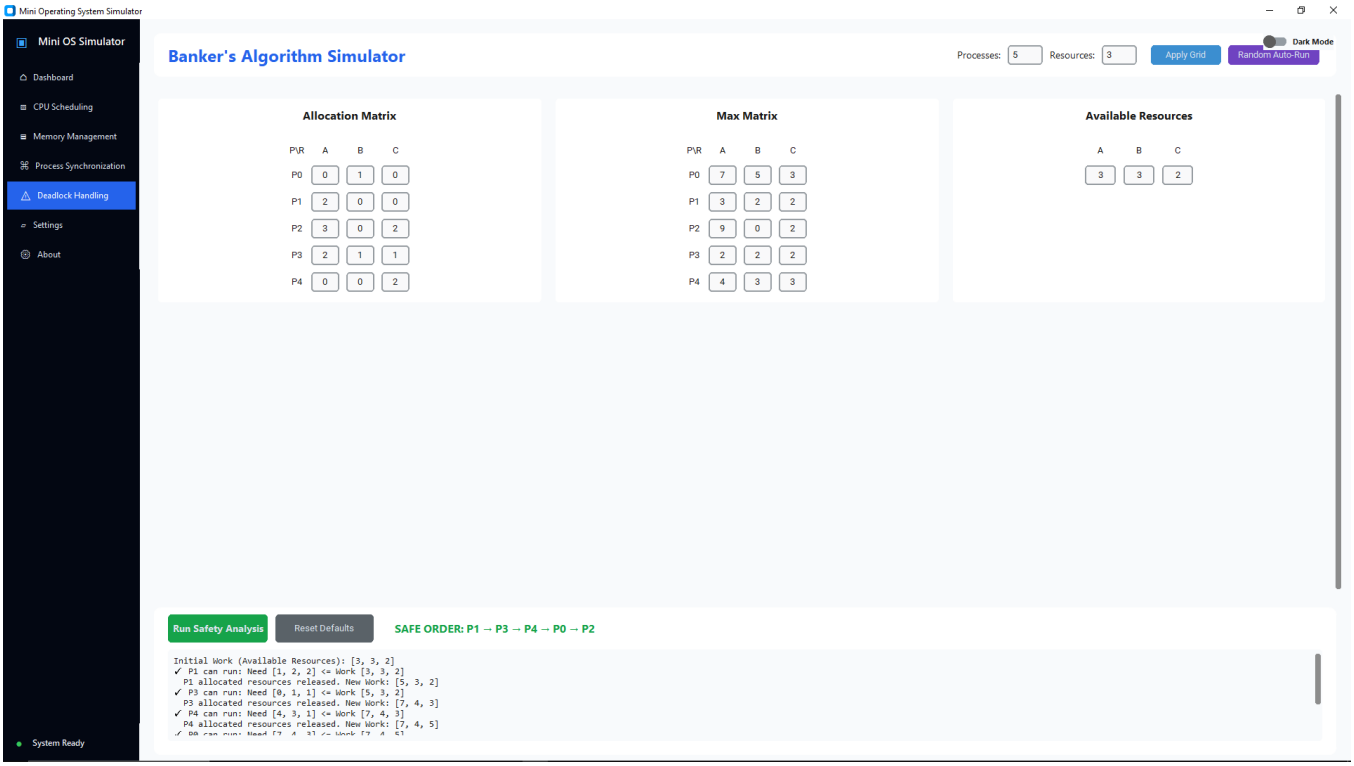


Figure 5: Banker's Algorithm resource table and generated safe sequence

8.1 Safety Algorithm

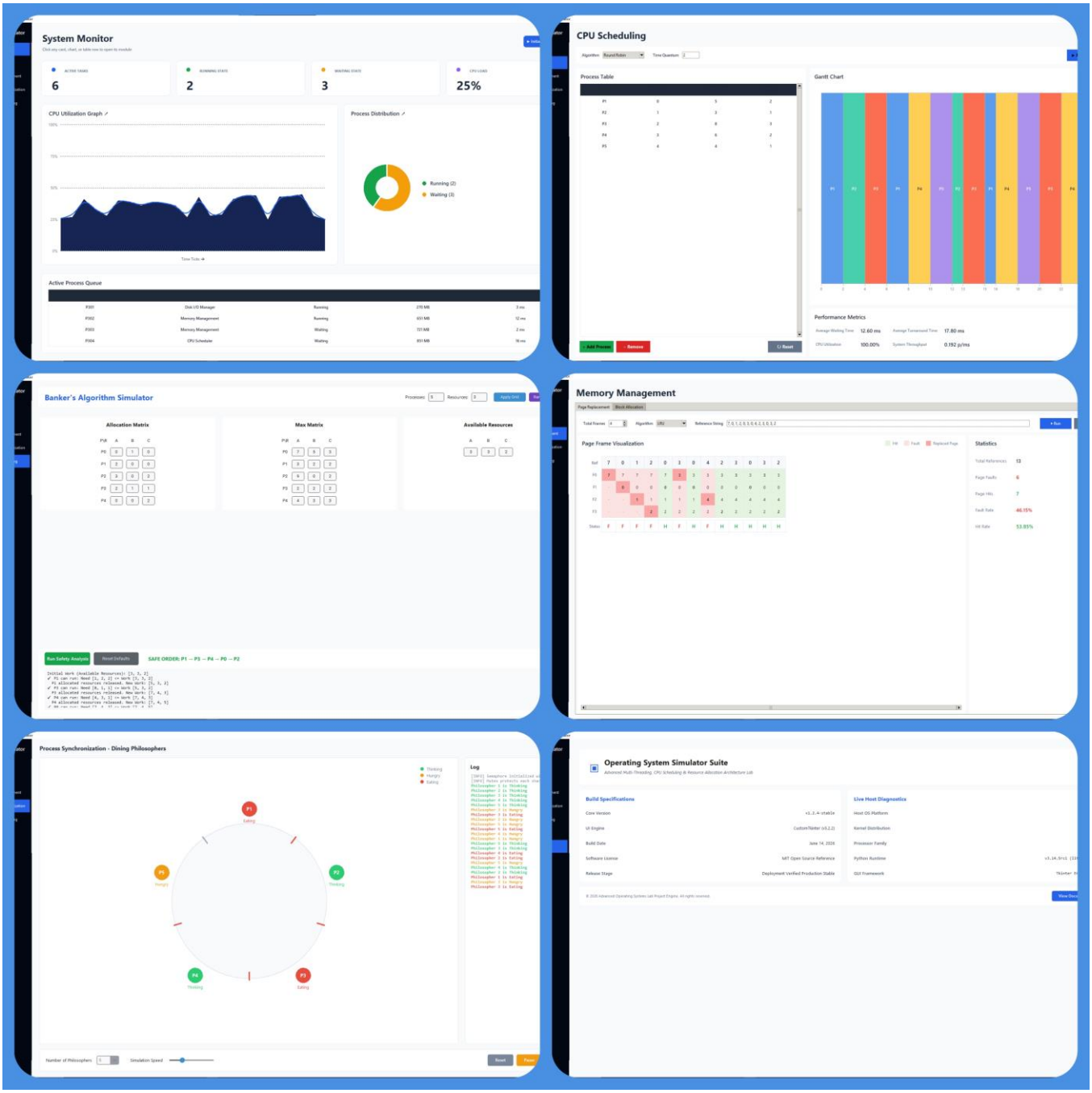
6. Copy Available into a temporary Work vector and mark every process unfinished.
7. Find an unfinished process whose Need is less than or equal to Work for every resource type.
8. Assume the selected process completes, then add its Allocation back to Work.
9. Record the process in the safe sequence and repeat until all processes finish or no process can proceed.
10. If every process finishes, the system is safe; otherwise a safe sequence does not exist and deadlock risk is reported.

Demonstration Result

For the default dataset, the simulator finds the safe sequence P1 → P3 → P4 → P0 → P2 and reports that the system is in a safe state.

Complete System Visualization

The following composite image was produced from actual program screenshots. It presents the six principal operating-system views in a format similar to the supplied visual reference.



Complete visual overview of the implemented Mini Operating System Simulator

9. Testing and Validation

The algorithm functions were executed independently from the GUI and the result lengths, completion values, fault counts, safe-state result, and non-negative waiting times were checked. The GUI was also launched under each module and screenshots were captured to confirm that the interface rendered successfully.

Test Case	Input / Action	Expected Result	Observed Result
TC-01	Run FCFS, SJF, Priority and RR on five processes	All processes receive completion and metric values	Passed
TC-02	Round Robin with quantum 2	Cyclic execution segments and complete Gantt chart	Passed
TC-03	Reference string with FIFO, LRU and Optimal	Frame history and fault/hit arrays match reference count	Passed
TC-04	Banker dataset with Available = [3,3,2]	A safe sequence containing all five processes	Passed
TC-05	Memory blocks and process sizes	Allocated/unallocated status and remaining space	Passed

Test Case	Input / Action	Expected Result	Observed Result
TC-06	Open every navigation module	All screens render without application failure	Passed

9.1 Sample Algorithm Validation Output

```
FCFS OK          average waiting time = 8.20
SJF OK           average waiting time = 6.60
Priority OK      average waiting time = 6.60
Round Robin OK   average waiting time = 12.60
FIFO faults = 7, LRU faults = 6, Optimal faults = 6
Banker safe sequence = [P1, P3, P4, P0, P2]
```

10. Installation and Execution

11.1 Required Software

- Python 3.10 or later
- Tkinter GUI package (included with most Windows Python installations)
- Pillow package for image support

11.2 Windows Execution

11. Extract the submitted ZIP file.
12. Open the mini_os_simulator folder.
13. Double-click start_simulator.bat, or open a terminal in the folder.
14. If running manually, install requirements and start the application using the commands below.

```
pip install -r requirements.txt
python app.py
```

Important :

Do not open app.py through a web browser. This is a desktop GUI application and must be executed with Python.

12. Limitations and Future Development

- The current simulator is educational and does not execute real operating-system processes.
- Process-table editing is intentionally simplified; a future version can add dedicated add/edit dialogs and persistent datasets.
- Synchronization is a controlled visualization rather than a benchmark of native operating-system threads.
- A future version can include segmentation, buddy allocation, Producer–Consumer, Reader–Writer, deadlock recovery, and exportable result reports.
- The interface can later be migrated to PyQt for richer animation and responsive layouts.

13. Conclusion

The Mini Operating System Simulator is a successful example of unifying the essential aspects of CPU scheduling, memory management, page replacement, synchronization of processes, deadlock prevention, and file allocation within one desktop application. The development of all six components contributes to the creation of an architecture that not only fulfills the basic requirements of system specifications but surpasses the regular standards.

The most remarkable accomplishment of the project includes the ability to integrate theoretical knowledge of algorithms into practical visualization tools. All complex system behavior can be analyzed using carefully selected visual hints:

- Gantt charts for chronological representation of CPU operations and context switching.
- Frame tables for demonstrating the exact functioning of page faulting.
- Philosopher state machines as visual aid to understanding synchronization.
- Safe sequences that prove the effectiveness of deadlock prevention.
- Colored blocks representing the layout of file allocation on disks.

In conclusion, it should be stressed that the simulator is an effective teaching instrument that provides opportunities for self-study and presentation purposes.

Appendix A: Submitted File Structure

```
OS-SIMULATOR/
├──Demo_Screenshots/
├── Python_File
              ├── dashboard.py
            ├── cpu_scheduler.py
            ├── page_replacement.py
            ├── process_sync.py
            ├── bankers_algorithm.pY
              ├── settings.py /
              ├── about.py
              |_main.py
```